Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Introducción a la Programación de Computadoras

Manual Técnico

Proyecto 1

Giovanni Josue Villegas Alvarez

202501086

ÍNDICE

INTRODUCCIÓN 3
DESCRIPCIÓN GENERAL DEL SISTEMA 4
ESTRUCTURA DEL PROYECTO 5
1. Modelo 5
2. Controlador 5
- ControlBiblioteca 5
- ControlUsuarios 6
- ControlLibros 6
- ControlPrestamos 7
3. Vista 7
4. Utilidades 8
5. Reportes 8
FUNCIONAMIENTO DEL SISTEMA 9
CLASES PRINCIPALES 10
1. ControlBiblioteca 10
2. ControlUsuarios 10
3. ControlLibros 10
4. ControlPrestamos 11
5. GeneradorReportes 11
MANEJO DE ARCHIVOS 12
VALIDACIONES IMPLEMENTADAS 13
PROBLEMAS ENCONTRADOS 14
MEJORAS FUTURAS 15
CONCLUSIÓN 16

INTRODUCCIÓN

La implementación de sistemas informáticos en distintos ámbitos ha permitido optimizar procesos, mejorar la organización de la información y reducir significativamente los errores asociados al manejo manual de datos. En el caso de las bibliotecas, la administración de usuarios, libros, préstamos y reportes representa una tarea fundamental para garantizar el orden, el control y la disponibilidad de los recursos bibliográficos. Cuando estos procesos se realizan de forma manual, pueden presentarse inconvenientes como pérdida de información, duplicidad de registros, demoras en las consultas y dificultades en el seguimiento de préstamos y devoluciones.

Por esta razón, el presente proyecto consiste en el desarrollo de un sistema de biblioteca utilizando el lenguaje de programación Java, orientado a facilitar la gestión de la información de manera eficiente y estructurada. Este sistema ha sido diseñado con el fin de administrar los principales procesos que se llevan a cabo dentro de una biblioteca, permitiendo registrar usuarios, controlar libros disponibles, gestionar préstamos y generar reportes que apoyen la organización interna. Además, incorpora una interfaz gráfica, lo que brinda una interacción más clara, sencilla y accesible para el usuario.

El propósito principal del sistema es modernizar y simplificar el control de la información bibliotecaria, evitando la dependencia de métodos tradicionales que suelen ser más lentos y propensos a errores. Mediante esta herramienta, se busca ofrecer una solución práctica que contribuya al manejo ordenado de los datos, a la consulta rápida de registros y al seguimiento adecuado de las operaciones realizadas dentro de la biblioteca.

Para el desarrollo del sistema se utilizaron arreglos estáticos y archivos de texto como base para el almacenamiento y manipulación de la información, cumpliendo con la restricción de no emplear estructuras dinámicas como ArrayList. Esta condición permitió fortalecer el uso de fundamentos esenciales de programación, especialmente en el manejo de datos mediante estructuras fijas, lógica de control y persistencia básica de información. En este sentido, el proyecto no solo representa una solución funcional para la administración de una biblioteca, sino también una aplicación práctica de los conocimientos adquiridos en programación orientada al desarrollo de sistemas.

DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema funciona mediante un proceso de inicio de sesión o login, en el cual el usuario debe ingresar sus credenciales para poder acceder. Una vez validada la información, el sistema identifica el tipo de usuario y, con base en ello, muestra un menú específico según sus permisos y funciones asignadas. Esto permite que cada usuario tenga acceso únicamente a las opciones que le corresponden.

Dependiendo del tipo de usuario, el sistema muestra un menú distinto:

- Administrador
- Operador
- Estudiante

Cada uno de los usuarios cuenta con permisos diferentes dentro del sistema, de acuerdo con el rol que desempeña y las funciones que le corresponden. Esto permite controlar el acceso a las distintas opciones disponibles, de manera que cada usuario solo pueda realizar las acciones autorizadas.

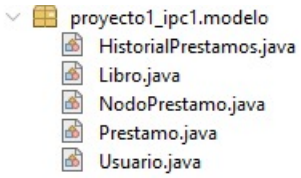
El sistema permite:

- Registrar y gestionar usuarios
- Registrar y gestionar libros
- Realizar préstamos y devoluciones
- Generar reportes

ESTRUCTURA DEL PROYECTO

El proyecto está organizado en paquetes para separar y estructurar mejor la lógica del programa:

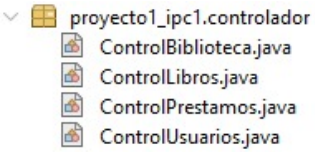
1. Modelo



Contiene las clases que representan los datos principales del sistema, es decir, las entidades con las que trabaja la aplicación durante su funcionamiento. En este apartado se definen las estructuras que almacenan la información de:

- Usuario
- Libro
- Préstamo

2. Controlador



Contiene la lógica principal del sistema, es decir, las clases encargadas de procesar la información y ejecutar las acciones necesarias para el funcionamiento de la biblioteca. En esta parte el proyecto posee 4 controladores:

- **ControlBiblioteca**

```java
package proyecto1_ipc1.controlador;

import proyecto1_ipc1.modelo.HistorialPrestamos;
import proyecto1_ipc1.modelo.Libro;
import proyecto1_ipc1.modelo.Prestamo;
import proyecto1_ipc1.modelo.Usuario;
import proyecto1_ipc1.reportes.GeneradorReportes;
import proyecto1_ipc1.utilidades.ArchivoManager;
import proyecto1_ipc1.utilidades.Bitacora;

/**
 *
 * @author Gio
 */
public class ControlBiblioteca {

    public static final int MAX_USUARIOS = 100;
    public static final int MAX_LIBROS = 200;
    public static final int MAX_PRESTAMOS = 500;

    private final Usuario[] usuarios;
    private final Libro[] libros;
    private final Prestamo[] prestamos;
    private final HistorialPrestamos[] historiales;

    private int totalUsuarios;
    private int totalLibros;
    private int totalPrestamos;

    private final ArchivoManager archivoManager;
    private final Bitacora bitacora;

    private final ControlUsuarios controlUsuarios;
    private final ControlLibros controlLibros;
    private final ControlPrestamos controlPrestamos;
    private final GeneradorReportes generadorReportes;

    public ControlBiblioteca() {
        usuarios = new Usuario[MAX_USUARIOS];
        libros = new Libro[MAX_LIBROS];
        prestamos = new Prestamo[MAX_PRESTAMOS];
        historiales = new HistorialPrestamos[MAX_USUARIOS];

        for (int i = 0; i < MAX_USUARIOS; i++) {
            historiales[i] = new HistorialPrestamos();
        }
```

- **ControlUsuarios**

```java
package proyecto1_ipc1.controlador;

import proyecto1_ipc1.modelo.Usuario;
import proyecto1_ipc1.utilidades.Validaciones;

/**
 *
 * @author Gio
 */
public class ControlUsuarios {

    private final ControlBiblioteca sistema;
    private String ultimoMensaje;

    public ControlUsuarios(ControlBiblioteca sistema) {
        this.sistema = sistema;
        this.ultimoMensaje = "";
    }

    public Usuario autenticar(String usuario, String contrasena) {
        if ("admin".equals(usuario) && "admin".equals(contrasena)) {
            sistema.getBitacora().registrar("LOGIN EXITOSO", usuario, "AUTENTICACION");
            return new Usuario(Usuario.ROL_ADMIN, "admin", "admin", "Administrador", "N/A");
        }

        Usuario encontrado = sistema.buscarUsuarioActivo(usuario);

        if (encontrado != null && encontrado.getContrasena().equals(contrasena)) {
            sistema.getBitacora().registrar("LOGIN EXITOSO", usuario, "AUTENTICACION");
            return encontrado;
        }

        sistema.getBitacora().registrar("LOGIN FALLIDO", usuario, "AUTENTICACION");
        return null;
    }

    public boolean registrarEstudiante(String carnet, String nombre, String carrera, String contrasena) {
        ultimoMensaje = "";

        if (Validaciones.textoVacio(carnet) || Validaciones.textoVacio(nombre)
                || Validaciones.textoVacio(carrera) || Validaciones.textoVacio(contrasena)) {
            ultimoMensaje = "Todos los campos son obligatorios.";
            return false;
        }

        if (sistema.buscarUsuarioActivo(carnet) != null || "admin".equalsIgnoreCase(carnet)) {
            sistema.getBitacora().registrar("OPERACION ERRONEA", carnet, "REGISTRO ESTUDIANTE");
            ultimoMensaje = "Ya existe un usuario con ese carné.";
```

- **ControlLibros**

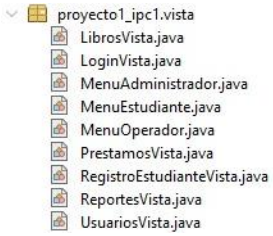
```java
package proyecto1_ipc1.controlador;

import proyecto1_ipc1.modelo.Libro;
import proyecto1_ipc1.utilidades.Validaciones;

/**
 *
 * @author Gio
 */
public class ControlLibros {

    private final ControlBiblioteca sistema;
    private String ultimoMensaje;

    public ControlLibros(ControlBiblioteca sistema) {
        this.sistema = sistema;
        this.ultimoMensaje = "";
    }

    public boolean registrarLibro(String codigo, String isbn, String titulo, String autor,
                                  String genero, int anio, int cantidad) {

        ultimoMensaje = "";

        if (Validaciones.textoVacio(codigo) || Validaciones.textoVacio(isbn)
                || Validaciones.textoVacio(titulo) || Validaciones.textoVacio(autor)
                || Validaciones.textoVacio(genero)) {
            ultimoMensaje = "Todos los campos son obligatorios.";
            return false;
        }

        if (!Validaciones.esIsbnValido(isbn)) {
            ultimoMensaje = "El ISBN debe tener exactamente 10 o 13 digitos.";
            return false;
        }

        if (cantidad <= 0) {
            ultimoMensaje = "La cantidad debe ser mayor que cero.";
            return false;
        }

        if (anio <= 0) {
            ultimoMensaje = "El año debe ser mayor que cero.";
            return false;
        }
```

- **ControlPrestamos**

```java
package proyecto1_ipc1.controlador;

import proyecto1_ipc1.modelo.Libro;
import proyecto1_ipc1.modelo.Prestamo;
import proyecto1_ipc1.modelo.Usuario;
import proyecto1_ipc1.utilidades.FechaUtil;

/**
 *
 * @author Gio
 */
public class ControlPrestamos {

    private final ControlBiblioteca sistema;
    private String ultimoMensaje;

    public ControlPrestamos(ControlBiblioteca sistema) {
        this.sistema = sistema;
        this.ultimoMensaje = "";
    }

    public boolean registrarPrestamo(String carnet, String codigoLibro) {
        ultimoMensaje = "";

        Usuario estudiante = sistema.buscarUsuarioActivo(carnet);

        if (estudiante == null) {
            ultimoMensaje = "El carné no existe.";
            sistema.getBitacora().registrar("OPERACION ERRONEA", carnet, "PRESTAMOS");
            return false;
        }

        if (!estudiante.esEstudiante()) {
            ultimoMensaje = "El usuario no es estudiante.";
            sistema.getBitacora().registrar("OPERACION ERRONEA", carnet, "PRESTAMOS");
            return false;
        }

        if (contarPrestamosActivos(carnet) >= 3) {
            ultimoMensaje = "El estudiante ya tiene 3 préstamos activos.";
            sistema.getBitacora().registrar("OPERACION ERRONEA", carnet, "PRESTAMOS");
            return false;
        }

        if (tienePrestamosVencidos(carnet)) {
            ultimoMensaje = "El estudiante tiene préstamos vencidos.";
            sistema.getBitacora().registrar("OPERACION ERRONEA", carnet, "PRESTAMOS");
            return false;
```

3. **Vista**

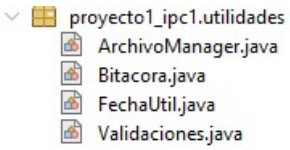
Contiene la interfaz gráfica del sistema, es decir, las ventanas, formularios, botones, menús y demás elementos visuales con los que el usuario interactúa. En esta parte se presenta la información de manera ordenada y se facilita el acceso a las distintas funciones del sistema:



- LoginVista
- MenuAdministrador
- MenuOperador
- MenuEstudiante
- UsuariosVista
- LibrosVista
- PrestamosVista
- ReportesVista
- RegistroEstudianteVista

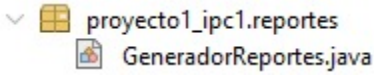
4. Utilidades

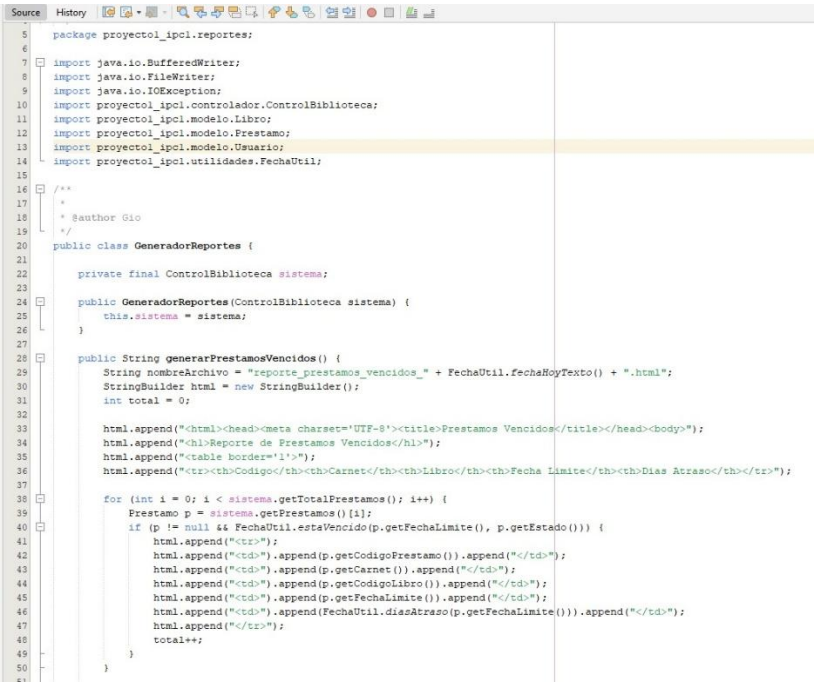
Contiene clases auxiliares que apoyan el funcionamiento general del sistema. En esta sección se incluyen métodos y recursos complementarios que facilitan tareas específicas:



- Manejo de archivos
- Validaciones
- Manejo de fechas
- Bitácora

5. Reportes





```java
package proyecto1_ipc1.reportes;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import proyecto1_ipc1.controlador.ControlBiblioteca;
import proyecto1_ipc1.modelo.Libro;
import proyecto1_ipc1.modelo.Prestamo;
import proyecto1_ipc1.modelo.Usuario;
import proyecto1_ipc1.utilidades.FechaUtil;

/**
 *
 * @author Gio
 */
public class GeneradorReportes {

    private final ControlBiblioteca sistema;

    public GeneradorReportes(ControlBiblioteca sistema) {
        this.sistema = sistema;
    }

    public String generarPrestamosVencidos() {
        String nombreArchivo = "reporte_prestamos_vencidos_" + FechaUtil.fechaHoyTexto() + ".html";
        StringBuilder html = new StringBuilder();
        int total = 0;

        html.append("<html><head><meta charset='UTF-8'><title>Prestamos Vencidos</title></head><body>");
        html.append("<h1>Reporte de Prestamos Vencidos</h1>");
        html.append("<table border='1'>");
        html.append("<tr><th>Codigo</th><th>Carnet</th><th>Libro</th><th>Fecha Limite</th><th>Dias Atraso</th></tr>");

        for (int i = 0; i < sistema.getTotalPrestamos(); i++) {
            Prestamo p = sistema.getPrestamos()[i];
            if (p != null && FechaUtil.estaVencido(p.getFechaLimite(), p.getEstado())) {
                html.append("<tr>");
                html.append("<td>").append(p.getCodigoPrestamo()).append("</td>");
                html.append("<td>").append(p.getCarnet()).append("</td>");
                html.append("<td>").append(p.getCodigoLibro()).append("</td>");
                html.append("<td>").append(p.getFechaLimite()).append("</td>");
                html.append("<td>").append(FechaUtil.diasAtraso(p.getFechaLimite())).append("</td>");
                html.append("</tr>");
                total++;
            }
        }
```

FUNCIONAMIENTO DEL SISTEMA

El sistema inicia en la pantalla de login, donde el usuario debe ingresar sus credenciales para que el sistema pueda validar su acceso correctamente.

Si el usuario es correcto, se redirige al menú correspondiente, mostrando únicamente las opciones permitidas según su tipo de usuario:

- Administrador → acceso completo a las funciones del sistema, incluyendo la gestión de usuarios, libros, préstamos, devoluciones y reportes.
- Operador → acceso limitado a las funciones asignadas para el control y manejo de las operaciones principales dentro de la biblioteca.
- Estudiante → solo consulta y préstamos, permitiéndole visualizar información disponible y realizar las acciones autorizadas dentro del sistema.

Los datos se cargan desde archivos de texto al iniciar el sistema y se guardan cuando se realizan cambios, lo que permite mantener la información actualizada y conservar los registros necesarios para el funcionamiento correcto de la biblioteca.

CLASES PRINCIPALES

1. **ControlBiblioteca**

Es la clase principal que conecta y coordina todo el sistema, funcionando como el eje central de la aplicación.

Se encarga de:

- Coordinar los demás controladores
- Manejar los arreglos de datos
- Proporcionar métodos a las vistas

2. **ControlUsuarios**

Se encarga de todo lo relacionado con los usuarios del sistema, administrando su acceso, registro y control dentro de la biblioteca.

- Login
- Registro de estudiantes y operadores
- Modificación de usuarios
- Eliminación de usuarios
- Validaciones

3. **ControlLibros**

Administra toda la información relacionada con los libros registrados en el sistema, permitiendo mantener un control ordenado del material bibliográfico.

- Registro de libros
- Modificación
- Eliminación
- Búsquedas

Incluye validaciones como:

- ISBN válido (10 a 13 dígitos)
- Control de duplicados

4. **ControlPrestamos**

Maneja todo el proceso relacionado con los préstamos de libros, permitiendo llevar un seguimiento adecuado de cada operación realizada por los usuarios.

- Registro de préstamos
- Devoluciones
- Control de estado (activo, vencido)

También controla que un estudiante no tenga más de 3 préstamos activos, con el fin de mantener el orden y cumplir con las reglas establecidas en el sistema.

5. **GeneradorReportes**

Genera reportes en formato HTML para presentar la información de manera clara, ordenada y fácil de consultar.

- Préstamos vencidos
- Libros disponibles
- Libros más prestados
- Estudiantes activos

MANEJO DE ARCHIVOS

El sistema utiliza archivos de texto para guardar la información necesaria para su funcionamiento, lo que permite almacenar los datos de manera sencilla, ordenada y accesible dentro del programa. Este método de almacenamiento fue implementado como una solución práctica para conservar la información generada por el sistema, evitando que los registros se pierdan al finalizar la ejecución. Además, el uso de archivos de texto facilita el manejo básico de datos y cumple con las condiciones establecidas para el desarrollo del proyecto.

cuentas.txt

Se utiliza para guardar la información relacionada con las cuentas de los usuarios del sistema, incluyendo los datos necesarios para el acceso y la identificación de cada tipo de usuario.

libros.txt

Almacena la información correspondiente a los libros registrados en la biblioteca, permitiendo conservar sus datos principales y mantener un control de los ejemplares disponibles.

prestamos.txt

Guarda los datos relacionados con los préstamos realizados, lo que permite llevar un seguimiento del historial, estado y control de devoluciones de los libros.

Esto permite mantener los datos, aunque el programa se cierre, ya que la información queda guardada de manera permanente y puede volver a cargarse cuando el sistema se inicie nuevamente. Gracias a ello, el sistema conserva la continuidad de los registros, facilita la actualización de la información y asegura un funcionamiento más estable en la administración general de la biblioteca.

VALIDACIONES IMPLEMENTADAS

Se implementaron varias validaciones dentro del sistema con el objetivo de prevenir errores, garantizar un manejo adecuado de la información y mantener un funcionamiento más seguro y ordenado en cada una de las operaciones realizadas. Estas validaciones son fundamentales, ya que permiten controlar los datos ingresados por los usuarios y evitar situaciones que puedan generar inconsistencias, fallos en los registros o problemas en la administración general de la biblioteca. Gracias a ello, el sistema puede operar de forma más confiable y con un mejor control sobre la información almacenada.

Campos vacíos

Se verifica que los formularios y registros no queden incompletos, evitando que se guarde información faltante o insuficiente dentro del sistema.

Datos duplicados

Se controla que no existan registros repetidos, lo que ayuda a mantener la información organizada y evita confusiones en la gestión de usuarios, libros o préstamos.

ISBN correcto

Se valida que el código ISBN de los libros cumpla con la longitud permitida, garantizando un registro más preciso y adecuado del material bibliográfico.

Límite de préstamos

Se restringe la cantidad de préstamos activos que puede tener un estudiante, asegurando el cumplimiento de las reglas establecidas dentro del sistema.

Restricciones al eliminar datos

Se impide eliminar información importante cuando esta puede afectar otros procesos relacionados, protegiendo así la integridad de los datos almacenados.

En conjunto, estas validaciones permiten fortalecer la seguridad del sistema, conservar la consistencia de la información y prevenir acciones incorrectas que puedan perjudicar su funcionamiento. Además, contribuyen a que el manejo de los datos sea más confiable, ordenado y eficiente dentro de la administración general de la biblioteca.

PROBLEMAS ENCONTRADOS

Durante el desarrollo del sistema se presentaron algunos inconvenientes que afectaban tanto la integración de sus componentes como el manejo correcto de la información. Sin embargo, estos problemas fueron identificados y corregidos de manera oportuna para mejorar la estructura, el funcionamiento y la estabilidad general del programa.

Errores al conectar las vistas con los controladores

Se solucionó unificando métodos, lo que permitió una mejor comunicación entre la interfaz gráfica y la lógica del sistema.

Nombres de métodos inconsistentes

Se estandarizaron, logrando una estructura más clara, ordenada y fácil de comprender dentro del código.

Problemas con guardado de datos

Se mejoró el manejo de archivos, permitiendo conservar la información de forma más segura y correcta.

Gracias a estas correcciones, el sistema alcanzó un funcionamiento más estable, organizado y eficiente durante su ejecución.

MEJORAS FUTURAS

Como parte de la evolución del sistema, se pueden considerar algunas mejoras que permitirían ampliar sus capacidades, optimizar su funcionamiento y brindar una experiencia de uso más completa. Estas posibles implementaciones contribuirían a que el sistema sea más flexible, eficiente y adaptable a necesidades más amplias dentro de la administración de una biblioteca.

Uso de ArrayList

→ Permitirá un manejo más dinámico de los datos, facilitando el almacenamiento, recorrido y modificación de la información dentro del sistema.

Base de datos en lugar de archivos

Mejorará la organización, seguridad y persistencia de los datos, permitiendo una administración más robusta y profesional de la información.

Interfaz más moderna

Hará que el sistema sea más atractivo, intuitivo y fácil de utilizar para los diferentes tipos de usuarios.

Mejor sistema de búsqueda

Facilitará la localización rápida y precisa de libros, usuarios o préstamos, mejorando la eficiencia en las consultas dentro del sistema.

Estas mejoras representan una oportunidad para seguir fortaleciendo el proyecto y convertirlo en una aplicación más completa, escalable y funcional.

CONCLUSIÓN

El sistema desarrollado cumple con los requerimientos establecidos para la gestión de una biblioteca, ya que permite administrar de forma organizada y eficiente la información relacionada con usuarios, libros, préstamos, devoluciones y reportes. A través de su funcionamiento, se logró reemplazar un control manual por una herramienta informática que facilita el manejo de los datos, mejora el orden en los procesos y reduce la posibilidad de cometer errores durante las operaciones principales del sistema.

Uno de los logros más importantes del proyecto fue la correcta implementación de la lógica del sistema junto con la interfaz gráfica, permitiendo que ambas partes trabajen de manera integrada y funcional. Esto hizo posible que el usuario interactúe con el programa de forma más clara y sencilla, mientras el sistema procesa y administra la información de acuerdo con las funciones correspondientes a cada rol. De esta manera, se consiguió una aplicación estructurada, comprensible y útil para el entorno bibliotecario.

Asimismo, la organización del proyecto en paquetes permitió mantener una estructura clara y ordenada dentro del desarrollo, separando adecuadamente los datos, la lógica del programa, la interfaz gráfica y las utilidades de apoyo. Esta distribución facilitó la comprensión del código, su mantenimiento y la corrección de errores, aportando mayor estabilidad al sistema. Además, el uso de validaciones ayudó a proteger la consistencia de la información, evitando problemas como datos vacíos, registros duplicados, préstamos fuera del límite permitido y errores al eliminar elementos importantes.

A pesar de las limitaciones establecidas, como el uso de arreglos estáticos y archivos de texto en lugar de estructuras dinámicas o bases de datos, el sistema logró funcionar correctamente y cumplir con su propósito principal. Esto demuestra que, mediante una buena organización de la programación y una lógica bien planteada, es posible desarrollar una solución funcional y eficiente utilizando herramientas básicas del lenguaje Java. En este sentido, el proyecto también representa una aplicación práctica de los conocimientos adquiridos en programación, especialmente en el manejo de arreglos, archivos, validaciones, modularización del código e interfaces gráficas.